

Báo Cáo Đánh Giá Kiến Trúc Mới (Web Application C&C Base)

1. Tổng quan & Lý do chọn kiến trúc này

1.1 Tổng quan

Kiến trúc của hệ thống đang được định hướng chuyển đổi từ mô hình ứng dụng điều khiển từ xa (Remote Control/RAT) giao tiếp trực tiếp kiểu cũ (thuần WinForms / TCP Socket) sang **Kiến trúc Web Application (Client-Server-Agent / C&C)**. Việc chuyển dịch này phân tách rõ ràng hệ thống thành các lớp đa tầng giao tiếp qua nền tảng HTTP/WebSockets, mở ra giải pháp triệt để cho các vấn đề kết nối trong môi trường mạng thực tế.

1.2 Lý do chọn kiến trúc này

- **Vượt rào Cổng mạng (NAT & Firewall bypass):** Khắc phục hoàn toàn điểm yếu rất lớn của kiến trúc cũ (Server.cs yêu cầu mở IP tĩnh ở cổng 5656 để Client.cs kết nối vào - điều gần như bất khả thi qua Internet do vướng NAT Router nội bộ của nạn nhân). Kiến trúc mới sử dụng kết nối ngược (Connect-back), tức là máy Target chủ động "gọi" ra ngoài Internet để gặp Backend Server, giúp bỏ qua mọi firewall.
- **Độ ổn định cao của WebSockets / SignalR:** Xử lý kết nối Real-time cực kỳ mượt và ổn định. Dễ dàng đóng gói các tệp định dạng lớn (như hình ảnh MemoryStream chụp màn hình) thành Base64 để gửi trên nền tảng Web tốt hơn việc bóc tách byte qua Raw TCP.
- **Mở rộng tính đa nền tảng cho bộ Điều khiển (Controller):** Bất cứ trình duyệt web nào (từ di động iOS/Android, MacOS cho đến máy tính công cộng) đều có thể đóng vai trò Master Controller mà không cần phải trải qua giai đoạn Download và run file Client.exe (WinForms) truyền thống.

2. Kiến trúc bao gồm những thành phần nào

Kiến trúc mới không còn chạy kiểu "1 đối 1" (Peer to Peer), mà được tổ chức thành bộ 3 thành phần chính:

- **Target Agent (Thay thế cho đoạn `server.cs` cũ):**
 - Chạy ngầm trên máy tính bị điều khiển, vẫn là một ứng dụng lập trình bằng C# .NET.
 - *Vai trò:* Nhận và thực thi các lệnh hệ thống từ OS (như truy xuất Process, Registry).
 - *Bản chất giao tiếp:* Từ bỏ việc mở Socket Server, nay trở thành một WebSocket Client tự động liên hệ tới Backend ngay khi khởi động.
- **Backend Server / Máy chủ C&C (Thành phần hoàn toàn MỚI):**
 - Xây dựng bằng ASP.NET Core Web API + công nghệ SignalR.
 - *Vai trò:* Là trung tâm nhận tín hiệu (Relay Station/Hub). Quản lý danh sách các Target Agent đang Online; Cung cấp địa chỉ kết nối API cho phía Web; Điều hướng truyền tải lệnh từ người quản lý đến trúng đích Agent yêu cầu.
- **Web Frontend (Thay thế cho giao diện `client.cs` WinForms cũ):**

- Viết bằng mã nguồn FrontEnd như ReactJS, VueJS hoặc Blazor WebAssembly chạy 100% trên trình duyệt. Tối ưu nhất cho ứng dụng web dạng điều khiển từ xa này sẽ là ReactJS
- *Vai trò:* Hiển thị màn hình Control Panel với các bảng, nút ấn trực quan UI/UX hiện đại để người vận hành (Admin) gửi thao tác hệ thống tới Backend.

3. Luồng hoạt động của kiến trúc (The Workflow)

Toàn bộ hành trình gọi một API điều khiển diễn ra qua chu trình 6 bước khép kín như sau:

1. **Agent khởi tạo:** Ngay khi Target Agent được bật ngầm, nó tạo một đường ống hàm mã hóa (WebSocket/SignalR tunnel) kết nối bền vững tới Backend Server.
2. **Cập nhật danh sách:** Backend Server đánh dấu Target Agent là "Online".
3. **Phát lệnh Admin:** Từ bất kỳ đâu, người quản trị mở đường link Web Frontend hiện trên Browser -> chọn Target Agent đó -> Click nút hành động (Ví dụ: "Lấy danh sách Process"). Yêu cầu này được đẩy bằng HTTPS từ UI đến REST API của Backend Server.
4. **Trung chuyển lệnh (Relay Command):** Backend Server nhận lệnh phân tích xem thuộc Agent ID nào -> Đẩy thông điệp yêu cầu GetProcess list xuống thẳng cho Target Agent thông qua đường hầm WebSocket có sẵn.
5. **Thực thi thiết bị:** Target Agent nhận lệnh, dùng thư viện native C# để kéo danh sách các tiến trình .exe đang chạy -> Đóng gói nội dung -> Phản hồi kết quả đẩy ngược lên lại cho Backend Server.
6. **Trả kết quả hiển thị:** Backend Server nhận được chuỗi dữ liệu kết quả, đẩy data này cho bảng Web Frontend để hiển thị lên UI màn hình cho Quản trị viên theo dõi.

4. Kiến trúc đó cho phép thực hiện gì?

Mọi tiềm năng trước đây ở bản cũ sẽ hoạt động trơn tru ở diện rộng tại bản mới. Nền tảng này hỗ trợ:

- **Giám sát Màn Hình (View Screen):** Stream hình ảnh màn hình thiết bị target Real-time thông qua giải mã chuỗi Base64 trên Web browser.
- **Giám sát Input (Keylogger):** Liên tục theo dõi thao tác gõ phím và truyền nhật kí text về cho trung tâm quản trị kiểm soát.
- **System Management:** Quản lý trọn gói các danh sách Process nội hàm, tương tác thao tác CRUD (tạo, xóa, sửa, đọc) trong cấu trúc Registry cực nhạy cảm của hệ điều hành.
- **Điều khiển Hàng loạt (Botnet Scale):** Một admin với một màn hình Web Dashboard duy nhất có thể quan sát, lọc và chỉ định hành động cùng lúc (Broadcast commands) cho hàng vạn thiết bị Target Agents từ nhiều khu vực địa lý khác nhau kết nối về Trung tâm điều hành duy nhất.

5. Những Thuận Lợi (Ưu điểm) Bổ Sung Của Mô Hình Web

- **Cập nhật và Bảo trì tập trung:** Khắc phục lỗi và ra mắt tính năng mới chỉ cần thực hiện một lần tại phía Backend Server và Frontend Code, mọi user quản trị sẽ ngay lập tức được cập nhật mà không cần gửi bản vá EXE.
- **Dễ dàng mở rộng quy mô (Scalability):** Backend thiết kế theo REST API vô cùng thuận tiện cho việc cắm thêm Load Balancers/Cloud Services khi lượng Target Agent bùng nổ mà kiến trúc Raw Socket cũ không chịu tải nổi.
- **Quản lý dữ liệu an toàn:** Log hoạt động, dữ liệu hình ảnh sẽ được chuyển hướng trích xuất vào CSDL Cloud (Database Server) bảo mật chống nguy mất mát ở client cục bộ.

6. Những Khó Khăn Và Thách Thức Nhất Định

- **Phụ thuộc kết nối Backbone của Backend:** Toàn bộ hệ thống sẽ tê liệt hoặc chết yểu tập trung nếu Backend Server trung gian bị sập do ngắt mạng, sập Hosting hoặc bị DDoS. Mọi Agent sẽ trong trạng thái "mất đầu".
- **Chi phí duy trì hạ tầng Cloud:** Cần có kinh phí thuê bao (Monthly/Yearly) cho Domain tĩnh, chứng chỉ bảo mật SSL/HTTPS bắt buộc, chi phí Database và Server Host/VPS chạy Backend API 24/7.
- **Thách thức lớn về các cấp độ Bảo mật:** Kiến trúc mở web API dễ bị scanner nhận diện tấn công (VD: tiêm mã XSS, SQL Injection). Buộc việc bảo mật JWT Auth Tokens tại Backend và Web UI phải xử lý cứng rắn.
- **Độ phức tạp nâng cao cho hệ thống code:** Đòi hỏi Developer phải chuyển từ logic code Desktop 1 cục (Monolithic WinForms) sang sự đa nhiệm của Fullstack: code Web Frontend (JS/React) kết hợp với Backend (C# SignalR/WebAPI) cũng như đồng bộ bất đồng bộ từ 3 nơi xa lạ nhau ghép lại.
- **Thách thức chuyển đổi (Migration):** Mất khá nhiều thời gian phân tích, test đập đi xây lại luồng xử lý luồng Buffer raw TCP stream cũ lên chuẩn cấu trúc event-driven mới của SignalR.